Generative Models for Protein-Protein Interaction Prediction

Paul Francis Jarabek Moore
Department of Computer Science
Western University
pmoore44@uwo.ca

Abstract

Protein-Protein interactions (PPIs) lie at the centre of many biological processes. Understanding and predicting these interactions is crucial in drug discovery, as experimental methods are often slow and expensive. Computational techniques have been in development for decades, but recent developments in deep learning have revolutionized PPI Prediction. This paper brings informed non-experts up to speed on the current developments in protein-protein interaction prediction following the deep learning revolution. It provides an in-depth background of relevant topics in machine learning and bioinformatics. Various approaches are reviewed, including methods that leverage sequence, structure, and network information, with an emphasis on generative models. These models include variational autoencoders, transformers, generative adversarial networks, and diffusion models.

1 Introduction

Proteins are essential macromolecules in all living organisms, playing a critical role in the structure, function, and regulation of the body's cells, tissues, and organs. These complex molecules are comprised of long chains of amino acids, which link together and fold into a unique, specific three-dimensional structure. The sequence and subsequent shape are crucial for determining the protein's particular function. Common protein roles include acting as enzymes to catalyze biochemical reactions (like digestion), providing structural support, enabling movement, serving as messengers, and protecting the body as antibodies [Reece et al., 2011].

Protein-protein interactions (PPIs) occur when physical contact is established between two or more proteins. PPIs facilitate chemical reactions, the formation of complex machineries, and signal pathways between cells [Morris et al., 2022]. Each of these is necessary for the essential functions of proteins. Consequently, PPIs can be targets for both diseases and drugs. Accurately predicting these interactions can accelerate the process of designing drugs by allowing cost-effective filtering of potential interactions with a target protein. Traditionally, PPIs are discovered using experimental techniques in a lab, using techniques like yeast 2 hybrid (Y2H) or co-immunoprecipitation (Co-IP). These methods are expensive and time-consuming, and sometimes inaccurate [Ding and Kihara, 2018]. For example, Co-IP can take multiple days [Tang and Takahashi, 2018], and Y2H has high throughput [Brückner et al., 2009], but has been known to yield results with high false positive rates [Vidalain et al., 2004].

There are three categories of PPI detection methods: *in vitro*, *in vivo*, and *in silico*. *In vitro*, the experiment is performed in a controlled environment, outside the organism. *In vivo*, the experiment is performed on the organism. The methods mentioned above belong to these two categories and are typically more expensive than *in silico*, where the experiment is performed in a digital simulation [Rao et al., 2014]. *In silico* techniques are particularly attractive with the recent advances in deep learning, allowing unprecedented performance on a variety of problems in bioinformatics, including PPI prediction [Zhang et al., 2024, Guo et al., 2024].

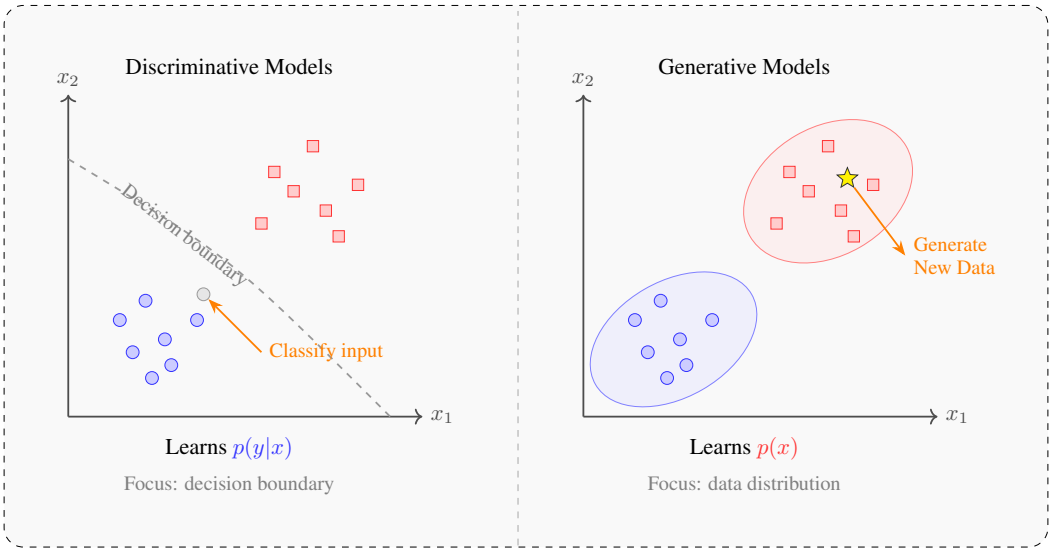

Figure 1: Generative vs. discriminative models.

Computational (*in silico*) methods for PPI prediction has been an area of interest for decades [Dandekar et al., 1998]. Starting with simple homology searches, and eventually making use of primary (amino acid sequence), secondary (local 3D structures), and tertiary structure (the entire 3D structure) of the proteins. As the field of machine learning advanced, so did PPI prediction techniques by leveraging the new power granted by increasingly sophisticated models.

The field of machine learning (ML) has undergone an unprecedented growth in recent years, primarily fueled by three factors: the availability of massive datasets (Big Data), a dramatic increase in computational power, and algorithmic breakthroughs (notably in deep learning with the rise of deep neural networks). This synergistic combination has enabled ML models to achieve near-human (and sometimes superhuman) performance in complex tasks like image recognition [Archana and Jeevaraj, 2024] and natural language processing [Qin et al., 2025].

Recently, the most advanced models are generative models, a revolutionary subclass of machine learning that focuses on learning the complex underlying data distribution to create entirely novel outputs rather than just classifying or predicting existing ones. For data features x and classification labels y, generative models learn the distribution $p(x)$ rather than the conditional probability $p(y|x)$, depicted in Figure 1. Models like generative adversarial networks [GANs; Goodfellow et al., 2014], variational autoencoders [VAE; Kingma and Welling, 2013], large language models [LLMs; Naveed et al., 2025], and diffusion models [Sohl-Dickstein et al., 2015] have demonstrated remarkable ability to synthesize realistic images, text, and biological structures [Ramesh et al., 2021, Radford et al., 2018, Abramson et al., 2024]. In the field of protein-protein interaction (PPI) prediction, generative models are excellent at understanding the hidden features of protein sequences and structures, facilitating state-of-the-art accuracy of predictions.

Previous work has surveyed multiple different areas of computational PPI prediction. Rao et al. [2014] provided a broad but superficial review of all methods at the time (over a decade ago), including *in vitro* and *in vivo* methods. Ding and Kihara [2018] classified the computational methods by which features of the proteins are used for prediction (sequence, structure, network, genome, function). Charih et al. [2025] covered only sequence-based approaches with a focus on methods that use the transformer architecture, and provided a review of real-world applications and deployments of such models. Kiouri et al. [2025a] covered only structure-based approaches, with a focus on modern deep learning (DL) models. Zhang et al. [2024] reviews advances in both structure prediction and interaction prediction. Taha [2025] provides a deeper background of the architectures being used and empirical testing for many models. However, none cover the application of generative models for predicting PPIs in-depth. This current work fills that research gap by first providing some background information on recent developments in generative AI, then a brief history of PPI prediction techniques using discriminative models like support vector machines [SVM; Cortes and

Vapnik, 1995] and convolutional neural networks [CNN; LeCun et al., 1989], among others. Finally, we describe the current state of the field, analyzing the most recent contributions to the field, including models that leverage the pivotal transformer architecture or the more recent discovery of diffusion models.

2 Background

This section provides all the necessary background information required to properly get up to speed on the current state-of-the-art in PPI prediction. We begin with proteins and PPIs, then datasets, and historical methods for PPI prediction. Next, we provide background on fundamental models in machine learning like neural networks [NNs; Rosenblatt, 1958], autoencoders, convolutional neural networks [CNNs; LeCun et al., 1989], recurrent neural networks (RNNs), long short-term memory [LSTMs; Hochreiter and Schmidhuber, 1997]. Then, word embedding models are discussed as well as their transferability to proteins. Finally, we discuss generative models like variational autoencoders [VAEs; Kingma and Welling, 2013], generative adversarial networks [GANs; Goodfellow et al., 2014], transformers [Vaswani et al., 2017], and diffusion models [Sohl-Dickstein et al., 2015].

2.1 Proteins

Proteins are the building blocks of life, serving as catalysts for countless crucial chemical reactions that occur in organisms, structurally support cells and tissues, hormones, enzymes, etc. [LaPelusa and Kaushik, 2022]. At the most basic level, proteins are composed of a chain of amino acids. Amino acids are molecules called organic compounds that are part of both the amino and carboxylic acid functional groups [Lehninger, 1970]. Of the hundreds of amino acids, only 20 common amino acids are used to create proteins, and an additional 2 are found in specific organisms or enzymes. Each of these 22 amino acids is assigned a single letter code (Table 1).

Table 1: The 22 amino acids used to form proteins, and their symbols. "*" denotes specialized amino acids found in specific organisms or enzymes.

Amino Acid	**3-Letter Code**	**1-Letter Symbol**
Alanine	Ala	A
Arginine	Arg	R
Asparagine	Asn	N
Aspartic Acid	Asp	D
Cysteine	Cys	C
Glutamic Acid	Glu	E
Glutamine	Gln	Q
Glycine	Gly	G
Histidine	His	H
Isoleucine	Ile	I
Leucine	Leu	L
Methionine	Met	M
Phenylalanine	Phe	F
Proline	Pro	P
Selenocysteine*	Sec	U
Serine	Ser	S
Threonine	Thr	T
Tryptophan	Trp	W
Tyrosine	Tyr	Y
Valine	Val	V
Pyrrolysine*	Pyl	O

Amino acids link to each other through peptide bonds and other molecular interactions, imposing various forces on the protein. These interactions result in the "folding" of the protein into a particular shape. This shape can vary slightly based on the environment and interactions with other proteins [Alberts et al., 2002].

2.2 Protein Interactions

Almost no proteins work in isolation; they need some other compound to interact with as a catalyst, or form a signal pathway, or complete some other function [Morris et al., 2022]. Inside a cell, proteins will collide with ligands (small molecules that bind to proteins), peptides (small chains of amino acids), or even other proteins. If the orientation of these compounds is correct, they will chemically bond to one another. These interactions are classified into 4 categories: transient, obligate, homo-oligomeric, and hetero-oligomeric. Transient interactions form and dissociate quickly. The interaction partners, therefore, exist in isolation most of the time. Obligate interactions form stable complexes between two compounds that rarely exist in isolation. Homo-oligomeric interactions occur between proteins and identical copies of themselves. They can form complexes with multiple copies at a time, not just two. Finally, hetero-oligomeric interactions form complexes with non-identical proteins.

Determining whether two given proteins will interact with each other at all is a fundamental problem in biology that has been extensively researched. Methods for solving this problem have been historically confined to a lab Akbarzadeh et al. [2024], but in recent years, with the advances in ML, computational methods have been gaining traction.

Computational methods for PPI prediction train ML models on large protein interaction and protein datasets (see Section 2.4) in order to accurately determine, when given two proteins (or a network of protein interactions), whether or not they will interact. Other problems in bioinformatics take a similar approach, such as binding affinity prediction [Stephenson and Karnati, 2025], where it is predicted how strong an interaction is between a protein and a ligand (typically), or interaction site prediction [Aumentado-Armstrong et al., 2015], where, given two proteins, the physical location of the interaction is predicted.

When predicting PPIs, there are three data sources to choose from: sequence, structure, or network. Sequence-based PPI prediction uses only the amino acid sequences of both proteins to determine whether or not they interact. Using only the sequence information is more efficient than using structural or network information, but is more challenging since the structure of a protein is a defining factor of its behaviour.

Structural information intrinsically includes sequence information, as it contains a 3D coordinate of each amino acid in the sequence. Avoiding the need to learn the structure of the protein from the sequence makes the problem easier, but may not be as applicable since structural information is more scarce and expensive to obtain than sequence. There has been some work to make standalone models for predicting the structure (AlphaFold [Abramson et al., 2024]), which could make structure-based approaches more viable.

Finally, one can predict PPIs based on known PPIs in the same interactome. This method is generally not applicable, as it learns nothing about the proteins themselves, and is not generalizable to interaction networks it did not see in training. This can be addressed by incorporating sequence, structure, or both into the prediction, but there will still be a tendency to learn the topology of the network rather than the intricacies of the proteins.

2.3 History

For computational PPI prediction, historically, features of protein sequences and structures were extracted via engineered methods [Charih et al., 2025]. Conjoint triad (CT) summarizes a protein sequence by grouping the 20 standard amino acids into seven classes based on their physicochemical properties and then counting the frequency of all possible $7 \times 7 \times 7 = 27$ triad combinations within the sequence [Shen et al., 2007]. Autocovariance (AC) captures the correlation or dependency between amino acids at a specific distance (lag) apart along the protein sequence based on their physicochemical properties, averaging this covariance over the entire sequence [Guo et al., 2008]. Amino acid composition (AAC) represents proteins as a fixed-length vector containing the frequency of each amino acid in the sequence. Pseudo amino acid composition (PseAAC) extends AAC to include some sequence order information by measuring the correlation of physicochemical properties of amino acids up to a certain λ apart [Chou, 2001]. Position-specific scoring matrix (PSSM) is generated by PSI-BLAST [Altschul et al., 1997] from multiple sequence alignments of homologous proteins and represents the conservation probability of each amino acid. These methods encode some important information about the sequences, and importantly, output a fixed-length representation of proteins regardless of their length, but are ultimately incapable of sufficiently capturing the complexity of biological systems individually. Such feature extraction methods were commonly used directly as input into a classical discriminative model. Shen et al. [2007] and Guo et al. [2008] both used SVMs as their classification method, with CT and AC as the feature extraction, respectively.

Later methods use representation learning, where an algorithm automatically discovers, or extracts, critical features from raw data. Methods like autoencoders, word embeddings, and deep neural networks (DNNs) achieve this.

Table 2: PPI Databases, the number of interactions they contain, and the number of unique sequences. Negatome 2.0 [Blohm et al., 2013] and Bernett et al. [2024] did not disclose the exact number of proteins.

Database	Reference	Interactions	Sequences
BioGRID	[Oughtred et al., 2021]	2,911,104	91,289
IntAct	[Orchard et al., 2013]	1,726,205	145,895
STRING	[Szklarczyk et al., 2022]	27,541,372,832	59,309,604
MINT	[Licata et al., 2011]	137,544	27,555
Mentha	[Calderone et al., 2013]	741,337	90,905
Negatome Database 2.0	[Blohm et al., 2013]	6,532	$\approx$ 1,900
Bernett Gold Standard	[Bernett et al., 2024]	274,500	$\approx$ 14,077

Table 3: Protein databases, the number of individual proteins they contain, and the dominant source of proteins.

Database	Reference	Proteins	Most represented
UniProt	[Consortium, 2024]	199,579,900	Homo sapiens
PDB (Experimental)	[Berman et al., 2000]	246,045	Homo sapiens
AlphaFold DB	[Bertoni et al., 2025]	241,070,489	Glycine max
RefSeq	[Goldfarb et al., 2024]	427,129,536	Bacteria

2.4 Datasets

Labeled datasets of interacting pairs of proteins are required to evaluate the accuracy of PPI prediction models. Databases like BioGRID, IntAct, and STRING provide millions of interacting pairs of proteins, but no non-interacting pairs. Such negative datasets are critical to the training of prediction algorithms [Ben-Hur and Noble, 2006]. There are negative datasets like Negatome DB 2.0 [Blohm et al., 2013] which contains only 6,532 examples in contrast with the massive sets of positive examples like BioGRID, IntAct, or STRING which contain millions (see Table 2).

For a generative model to learn the language of proteins and be able to properly extract information from individual proteins, the labeled interaction data is not necessary. Furthermore, databases like STRING [Szklarczyk et al., 2022] do not always include information on the tertiary structure of the proteins. Therefore, separate databases, which contain individual protein data like primary structure (UniProt [Consortium, 2024], RefSeq [Goldfarb et al., 2024]) or tertiary structure (PDB [Berman et al., 2000], AlphaFold DB [Bertoni et al., 2025]) are used. Some common protein databases are presented in Table 3.

A common issue among published PPI prediction methods is data leakage from randomly splitting such datasets into train/test partitions [Bernett et al., 2024]. Bernett et al. [2024] conducted a study evaluating many of the state-of-the-art models on datasets specifically crafted to determine whether the models are truly learning the underlying patterns of protein-protein interaction, or if they are relying on sequence similarity or node degree information shortcuts (hub proteins). They concluded that such shortcuts are entirely responsible for the high accuracy in the literature. As a result of these findings, Bernett et al. [2024] created a new gold standard dataset for evaluating sequence-based PPI prediction models that ensures the absence of data leakage and released it publicly.

2.5 Neural Networks

Most modern machine learning algorithms today are based on the idea of a neural network: a concept originally inspired by the biology of brains. The first of which was pioneered by Rosenblatt [1958] with the perceptron, a single-layer neural network. One drawback of the original perceptron was its inability to learn to classify non-linearly separable data. Rumelhart et al. [1986] introduced the backpropagation algorithm, which allowed easy training of multi-layer perceptrons, which could be fitted with non-linear activation functions. This solved the original perceptron's problem, but introduced new issues like vanishing gradient, lack of computational power, and limited data. In recent years, each of these issues has been more or less solved with huge amounts of computational

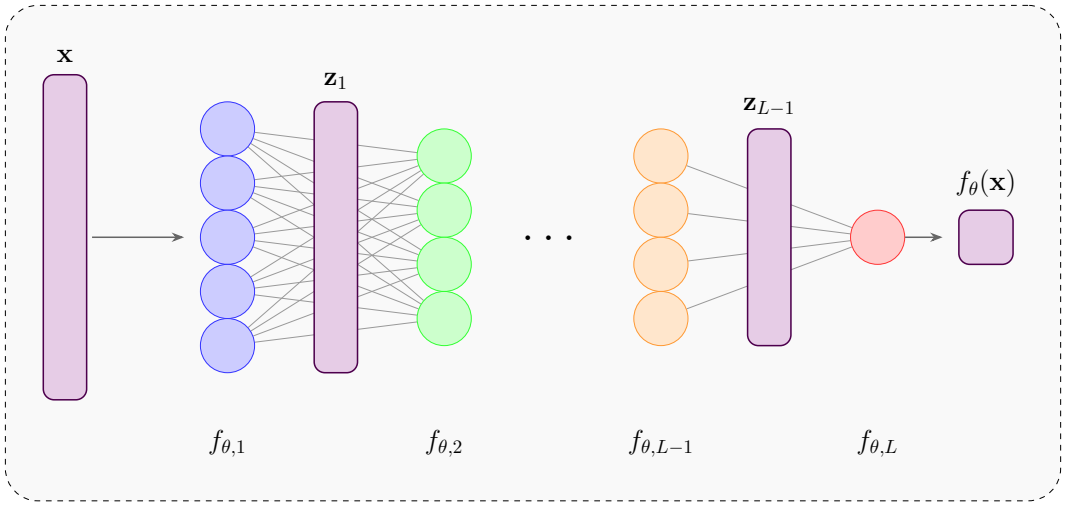

Figure 2: The general architecture of a neural network with L layers.

power, data, and algorithmic innovations like the ReLU activation function or dropout. With more computing power, data, and a wider variety of algorithms came DNNs. DNNs are fundamentally based on MLPs, but have a large number of hidden layers.

A DNN f_θ can be expressed as a composition of layers $f_\theta^l : \mathbb{R}^{d_{l-1}} \to \mathbb{R}^{d_l}$ and non-linear functions $h : \mathbb{R} \to \mathbb{R}$ applied component-wise:

$$f_\theta(x) = f_{\theta,L}(h(f_{\theta,L-1}(...(f_{\theta,1}(x))...)))$$

where L is the number of layers in the network. Each function/layer $f_{\theta,l}$ has it's own set of parameters θ, which govern the weights $\mathbf{W}$ and bias $\mathbf{b}$ of each neuron. For example, a fully connected layer (each neuron takes as input the output of each of the previous layer's neurons, see Figure 2):

$$f_{\theta,l}(\mathbf{z}_{l-1}) = \mathbf{W}_l \mathbf{z}_{l-1} + \mathbf{b}_l$$

$$\mathbf{z}_l = h(f_{\theta,l}(\mathbf{z}_{l-1}))$$

Training neural networks is done by minimizing a loss function. Simply put, a loss function measures the distance from the model's output to the correct value. For example, a common loss function is the mean squared error (MSE) loss:

$$\mathcal{L}(\theta, x, y) = \frac{1}{N} \sum_{i=1}^{N} (f_\theta(x_i) - y_i)^2$$

The output of the penultimate, or final hidden layer $\mathbf{z}_{L-1}$ is the internal representation of the input x, which is used for whatever task is required, be it classification or regression.

2.6 Autoencoders

Autoencoders learn to compress data into a compressed latent space, then reconstruct the original data with minimal loss (see Figure 3). This requires the latent space to preserve the most important features. An autoencoder is comprised of two neural networks: the encoder $\text{Enc}_\theta : \mathcal{X} \to \mathbb{R}^K$, and the decoder $\text{Dec}_\phi : \mathbb{R}^K \to \mathcal{X}$ where $\mathcal{X}$ is the set of all possible inputs, and K is the number of dimensions for the latent vectors.

Both networks are typically trained in tandem, for example, by minimizing the total reconstruction loss

$$\arg\min_{\theta,\phi} \frac{1}{2} \sum_{n=1}^{N} ||x_n - \text{Dec}_\phi(\text{Enc}_\theta(x_n))||_2^2$$

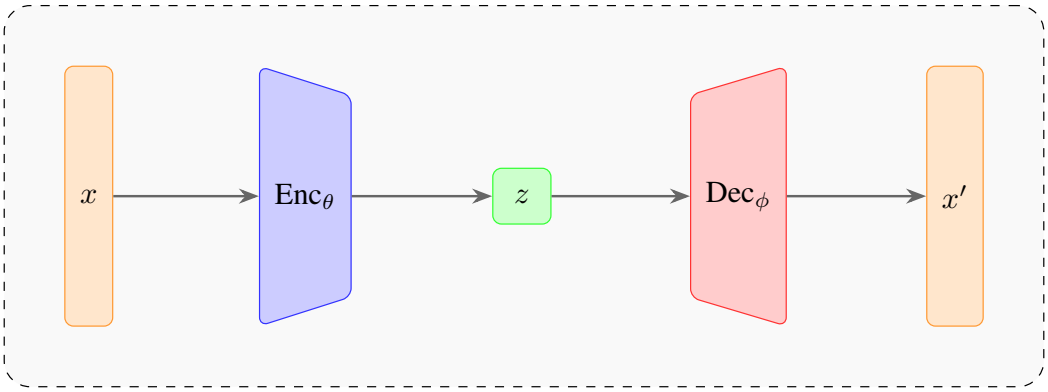

Figure 3: The general architecture of an autoencoder, with an encoder parametrized by θ, latent dimension K, and decoder parametrized by ϕ.

While autoencoders work for learning a compressed representation of the input, they are not capable of generating new data. That is, they were not designed to learn the underlying distribution $p(x)$. Variational autoencoders [Kingma and Welling, 2013] address this issue, and are covered in section 2.12.2

In the context of PPI prediction, Sun et al. [2017] is an example of the use of autoencoders. Their model used CT and AC as input for a stacked autoencoder (SAE) to learn features, and added a softmax classifier head for the final output.

2.7 Convolutional Neural Networks

CNNs are a class of deep learning models specifically designed to process grid-like data, most notably images. Traditional neural networks struggle with the high dimensionality of such data, but CNNs excel, identifying patterns like edges, textures, and, with enough layers, complex objects.

A CNN typically consists of three components: convolution, pooling, and classification/regression. The convolutional component uses a kernel matrix that slides across the input, performing some operation (like dot product) at each position. There may be many convolution layers in the network, each serving to detect different features depending on the kernel. The output of the convolution layer for 2D input (like an image), often called the feature map, is defined as

$$\mathbf{F}^{I\times J}(i,j) = f_{i,j} = \sum_{m=1}^{M}\sum_{n=1}^{N} x_{i+m,j+n}k_{m,n}$$

where $x_{i+m,j+n}$ is the value of the input matrix $\mathbf{X}^{I+m\times J+n}$ at position $i+m, j+n$ and $k_{m,n}$ is the kernel matrix $\mathbf{K}^{M\times N}$ at position m, n. Since the feature map is always smaller than the input matrix, the input must be padded if it is required to maintain its original size. For simplicity, we say that

$$\mathbf{F} = \text{conv}(\mathbf{X})$$

Pooling downsamples the output of the convolutional layers in order to reduce computational complexity without compromising any important features of the image [Zafar et al., 2024]. Pooling works by combining in some way (e.g., average, max, min) segments of the data. Max pooling can be defined as

$$\mathbf{Y}_{i,j} = max_{m,n\in\mathbf{W}_{i,j}}\mathbf{X}_{m,n}$$

where $\mathbf{Y}_{i,j}$ is the output after pooling at position i, j, $\mathbf{W}_{i,j}$ is the set of indicies of the input $\mathbf{X}$ for the region covered by the pooling window. Finally, the multi-dimensional data can be flattened into a vector and used as input for some NN, which will perform the final classification or regression. As a shorthand, we say

$$\mathbf{Y} = \text{maxpool}(\mathbf{X})$$

A recent example of a PPI prediction model that utilized CNNs is Chen et al. [2019]'s PIPR. In particular, they pass pre-trained Skip-Gram (based on Word2Vec Mikolov et al. [2013]) embeddings into an array of CNNs, and then an array of RNNs. The resulting embeddings from each protein are multiplied element-wise and passed to an MLP for classification. PIPR was also trained on the tasks of binding affinity and interaction type prediction.

2.8 Word Embeddings

Natural language processing (NLP) is the field at the intersection of computer science and linguistics. Its primary goal is to enable computers to understand, interpret, generate, or process human language in a useful way. All modern LLMs like OpenAI's GPT [Brown et al., 2020] or Google's Gemini [Team et al., 2025] are incredibly capable in each of these aspects.

Static word embedding methods like Word2Vec [Mikolov et al., 2013] or FastText [Bojanowski et al., 2016] train on a large corpus to predict context words (words that appear around a given target word). The embeddings are contained in the final learned weight matrix connecting the input layer to the hidden layer. The embedding for each word contains useful semantic information.

One classic example for how word embeddings work is the King $-$ Man $+$ Woman $\approx$ Queen equation [Mikolov et al., 2013](Figure 4), which demonstrates at least some understanding of each of the words as well as their interaction with one another. This understanding allowed models like Word2Vec to be applied to a variety of problems in linguistics, like part-of-speech (POS) tagging, named entity recognition (NER), or sentiment analysis [Sivakumar et al., 2020].

One shortcoming of such models is their inability to capture the context of the words. In natural language, the meaning of a word changes depending on the surrounding words in the sentence, so assigning a single, static vector to every instance of a word does not make sense. For example, in the sentences:

I ate an **orange**

My car is **orange**

The word orange has two different meanings. The first is a noun representing the fruit, and the second is used as an adjective for a car. While keeping the deployment cost of such models low (since you only need to compute the embeddings once) is nice, context is important for most aspects of natural language. Thus, more modern embedding models like the LLMs mentioned above, or simpler transformer (see Section 2.12.1) based models like BERT [Devlin et al., 2018], T5 [Raffel et al., 2020], or XLNet [Yang et al., 2019] capture the context of each instance of a word at the cost of more computational cost as the entire sentence must be used as input each time an embedding is generated.

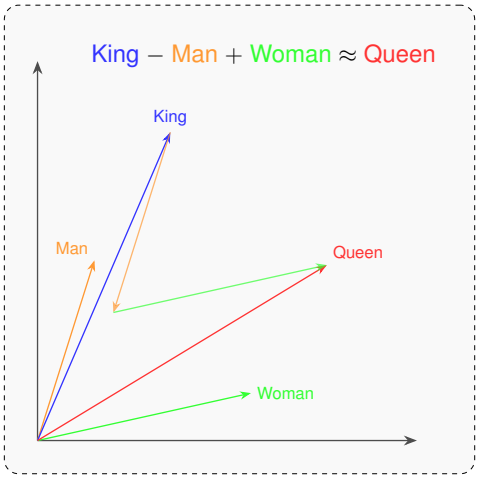

Figure 4: The classic King $-$ Man $+$ Woman $\approx$ Queen example [Mikolov et al., 2013] illustrated.

A common technique for training a contextual word embedding model is masked language modeling (MLM). MLM is a denoising objective that has the model predict tokens that were randomly masked or corrupted. Essentially, the model learns to "fill in the blanks". The output of the final hidden layer $\mathbf{z}_{L-1}$ (recall Section 2.5) can be extracted and used as a contextually rich word embedding.

2.9 Protein Embeddings

Clearly, word embeddings are a powerful tool for NLP, and demonstrate that a model can learn the underlying patterns and behaviours of a complex system like a human language. So why not do the same with the "language" of the human body?

A protein $\mathbf{p}$ of length N can be represented as a vector of amino acids (residues):

$$\mathbf{p} = [a_1, a_2, ..., a_{N-1}, a_N]^\intercal$$

If we consider each amino acid to be a word, and our protein sequence a sentence, then it should be possible to learn representations of proteins in the same way that word embedding models do. Indeed, embedding models based on many word embedding models started appearing. For example, Asgari and Mofrad [2015] proposed ProtVec based on Word2Vec, ProtBert [Brandes et al., 2022] based on BERT, or ProtT5 [Elnaggar et al., 2022] based on T5 [Raffel et al., 2020]. These protein embedding models have demonstrated similar ability in learning both natural language and the "language of proteins" [Tran et al., 2023]. The transformer-based protein embedding models are sometimes referred to as protein language models (PLMs).

2.10 Generative models

In this section, we present the intuition for, and define some generative model architectures. We begin with models designed for handling and generating sequential data: RNNs, LSTMs, and transformers. Then, models designed to generate convincing new data in a single pass: VAEs, GANs. Finally, we discuss diffusion models, which iteratively refine noise until it resembles the original data.

2.11 Recurrent Neural Networks

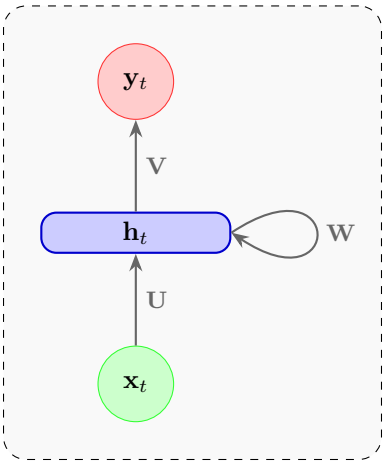

Figure 5: The "rolled" representation of a RNN. Input $\mathbf{x}_t$ is read for each time-step t, multiplied by the input weight matrix $\mathbf{U}$, the hidden state is updated using the hidden state weight matrix $\mathbf{W}$, and the output is calculated using the updated hidden state and the output weight matrix $\mathbf{V}$.

RNNs are a type of NN that are specifically designed to handle sequential data, particularly where data points depend on the context (points found earlier in the sequence). For example, a simple RNN known as an Elman Network [Elman, 1990], the network maintains a hidden state $\mathbf{h}$ as it reads each data point of an input one at a time. Each hidden state is calculated using the current input at time-step t and the previous hidden state:

$$\mathbf{h}_t = f_h(\mathbf{U}\mathbf{x} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

where f_h is the activation function, $\mathbf{U}, \mathbf{W}$ are the learned weights for the input and hidden state, respectively, $\mathbf{b}_h$ is the bias, and $\mathbf{x}$ is the input. The output of the layer is calculated using the freshly updated hidden state:

$$\mathbf{y}_t = f_y(\mathbf{V}\mathbf{h}_t + \mathbf{b}_y)$$

where f_o is the activation function (not necessarily the same as the one for the hidden state), $\mathbf{V}$ is the learned weights, and $\mathbf{b}_y$ is the bias. RNNs suffer from the vanishing gradient problem, where the gradient shrinks rapidly over the time-steps, resulting in the "forgetting" of long-term dependencies. One architecture that addresses the vanishing gradient problem in RNNs is the LSTM [Hochreiter and Schmidhuber, 1997], see Section 2.12.

2.12 Long Short Term Memory

LSTMs are an extension of RNNs and are designed to combat the vanishing gradient problem. The intuition behind them is to use a learned mechanism to decide when the hidden state should be updated or reset, rather than continuously updating it at every time-step. This means that important information will be prioritized regardless of its position in the sequence of inputs.

This is achieved by adding an internal cell state governed by three "gates": the input gate, the forget gate, and the output gate. The internal cell state acts as the node's long-term memory. The input gate determines whether or not the current input and hidden state should affect the internal state.

The forget gate determines whether or not the internal cell state should be reset (wiping the long-term memory). The output gate determines how much the internal cell state should affect the output (hidden state/short-term memory).

Each gate uses the logistic-sigmoid σ activation function, so the output is between 0 (let nothing through) and 1 (let everything through). The input $\mathbf{i}_t$, forget $\mathbf{f}_t$ and output $\mathbf{o}_t$ gates are calculated by:

$$\mathbf{i}_t = \sigma(\mathbf{U}_i\mathbf{x}_t + \mathbf{W}_i\mathbf{h}_{t-1} + \mathbf{b}_i)$$
$$\mathbf{f}_t = \sigma(\mathbf{U}_f\mathbf{x}_t + \mathbf{W}_f\mathbf{h}_{t-1} + \mathbf{b}_f)$$
$$\mathbf{o}_t = \sigma(\mathbf{U}_o\mathbf{x}_t + \mathbf{W}_o\mathbf{h}_{t-1} + \mathbf{b}_o)$$

where $\mathbf{U}$ and $\mathbf{W}$ are weights, $\mathbf{b}$ are the biases, $\mathbf{x}$ is the input, and $\mathbf{h}_{t-1}$ is the hidden state. Then, the internal cell state C_t is updated:

$$C_t = \mathbf{f}_t \odot C_{t-1} + \mathbf{i}_t \odot \tilde{C}_t$$

where $\tilde{C}_t$ are the potential changes to the internal state, known as the input node, and calculated at time-step t with weights $\mathbf{U}_c$ and $\mathbf{W}_c$, and hidden state $\mathbf{h}_{t-1}$:

$$\tilde{C}_t = \tanh(\mathbf{U}_c\mathbf{x}_t + \mathbf{W}_c\mathbf{h}_{t-1} + \mathbf{b}_c)$$

Finally, the hidden state is updated:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(C_t)$$

and both the hidden and internal states are used as input for the next time-step.

2.12.1 Transformers

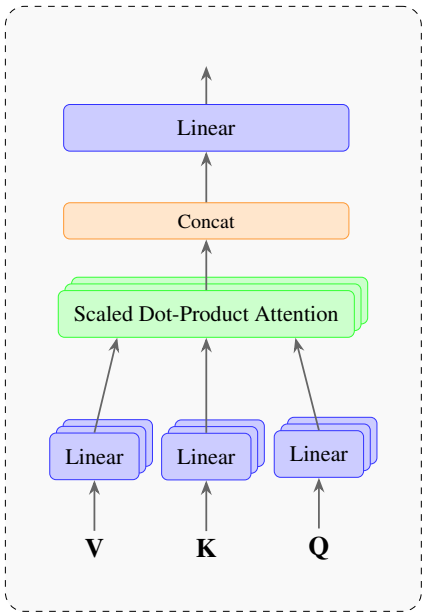

Figure 6: Multi-head attention module [Vaswani et al., 2017]. Each layer in the "stack" of linear projections projects the input with different weights, then passes them through the attention module, and finally concatenates them back together. The final linear projection returns the data to the original input dimension.

The transformer architecture was introduced in 2017 by Vaswani et al. [2017], and quickly became the foundation for many state-of-the-art models in NLP like GPT [Radford et al., 2018] and BERT [Devlin et al., 2018]. The transformer is based on an encoder-decoder structure, but with self-attention calculated in parallel via multiple "heads". Self-attention assigns weights to each token relative to each other, capturing important contextual information.

In the original transformer, attention is described as a mapping of queries, keys, and values to an output. The actual attention operation compares the queries and keys using dot product, then scaling, then softmax to determine the weights for each value. All queries, keys, and values are organized into their own matrices so that the attention can be computed on a set of queries simultaneously. The output is then just the weighted values:

$$\text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\mathbf{V}$$

The original transformer uses multi-head attention, where the input of dimension d_m is a linearly projected concatenation of multiple attention "heads" whose queries, keys, and values

are all projected with unique weight matrices to dimensions d_k, d_k, d_v respectively, allow the model to attend to different features simultaneously (see Figure 6). More formally, multi-head attention can be defined as:

$$\text{multihead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^{\mathbf{O}}$$

where $\text{head}_i = \text{attention}(\mathbf{Q}\mathbf{W}_i^{\mathbf{Q}}, \mathbf{K}\mathbf{W}_i^{\mathbf{K}}, \mathbf{V}\mathbf{W}_i^{\mathbf{V}})$ with $\mathbf{W}_i^{\mathbf{Q}} \in \mathbb{R}^{d_m \times d_k}, \mathbf{W}_i^{\mathbf{K}} \in \mathbb{R}^{d_m \times d_k}, \mathbf{W}_i^{\mathbf{V}} \in \mathbb{R}^{d_m \times d_v}$, and $\mathbf{W}^{\mathbf{O}} \in \mathbb{R}^{hd_v \times d_m}$.

2.12.2 Variational Autoencoders

A VAE combines the structure of a traditional autoencoder [Bank et al., 2023] with variational inference and probabilistic modeling (Figure 7). It is designed to translate the input into a distribution within its compressed, continuous, and structured internal representation known as the latent space, which allows it to generate new, similar data points through its decoder by sampling from the latent space and adding noise [Kingma and Welling, 2013].

Assume the data generation process consists of sampling some latent vector $\mathbf{z}$ from the latent distribution $p(\mathbf{z})$, then generating data $\mathbf{x}$ from a conditional distribution $p_\theta(\mathbf{x}|\mathbf{z})$ parametrized by the decoder. To train the model we need to find the parameters θ that maximize the likelihood of $\mathbf{x}$:

$$p(\mathbf{x}|\theta) = p_\theta(\mathbf{x}) = \int_{\mathbf{z}} p_\theta(\mathbf{x}|\mathbf{z})p_\theta(\mathbf{z})d\mathbf{z}$$

But this integral is intractable because it requires integrating over the large space of all $\mathbf{z}$. Since we can't compute $p_\theta(\mathbf{x})$, we can't compute the posterior $p_\theta(\mathbf{z}|\mathbf{x})$ either. Therefore, we introduce an approximate posterior parametrized by the encoder $q_\phi(\mathbf{z}|\mathbf{x})$. The loss function for training these networks (determining θ and ϕ) is negative evidence lower bound (ELBO):

$$\mathcal{L}(\theta, \phi) = -\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})] + D_{\text{KL}}[q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})]$$

where D_{KL} is the Kullback-Leibler (KL) Divergence. KL Divergence is a measure of the difference between two distributions p and q. Specifically, it is the difference between cross-entropy and entropy:

$$D_{\text{KL}}(p(z)||q(z)) = \int \log \frac{p(z)}{q(z)} p(z)dz = \mathbb{E}_{p(z)}\left[\log \frac{p(z)}{q(z)}\right]$$

Some properties of KL Divergence are that, for all distributions p, q it is non-negative ($D_{\text{KL}}(p||q) \geq 0$), asymmetric ($D_{KL}(p(z)||q(z)) \neq D_{\text{KL}}(q(z)||p(z))$), equal to 0 if and only if $p(z)$ and $q(z)$ are equivalent, and does not satisfy the triangle inequality. Since KL Divergence is non-negative, $\mathcal{L} \leq p(x)$. $p(x)$ is also called evidence, hence the name ELBO.

Finally, the loss function involves random sampling, which makes backpropagation impossible. To avoid this, instead of outputting $\mathbf{z}$ directly, the encoder outputs the mean μ and variance σ^2 from which we can determine $\mathbf{z}$:

$$\mathbf{z} = \mu + \sigma \odot \epsilon$$

where ϵ is a random noise vector. With these modifications, the encoder and decoder can be optimized using gradient descent.

2.12.3 Generative Adversarial Networks

A GAN is composed of two competing neural networks: a generator and a discriminator. They are trained simultaneously in an "adversarial" (or zero-sum game) process to generate new data that is indistinguishable from real data [Goodfellow et al., 2014].

The generator $g_\theta(\mathbf{z})$ learns parameters θ to map a distribution $p(\mathbf{z})$ of noise to the data space $p(\mathbf{x})$. g is trained to minimize $\log[1 - d_\phi(g_\theta(\mathbf{z}))]$. Simultaneously, the discriminator $d_\phi(\mathbf{x})$ is trained to

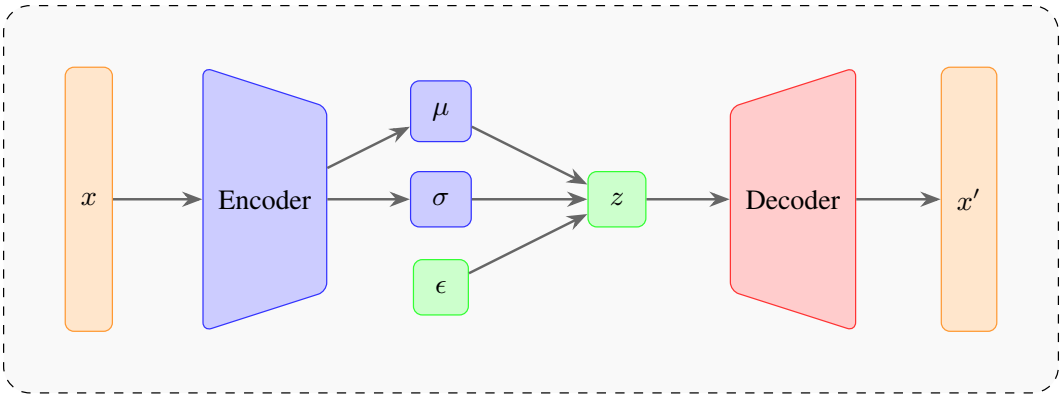

Figure 7: An illustration of the VAE architecture with the reparameterization trick for enabling backpropagation. Here, the encoder compresses the input x into μ and σ, which parameterize the latent space. z is sampled from that distribution using noise from ϵ, then passed as input to the decoder. The decoder attempts to recreate x as closely as possible.

maximize the probability of correctly labeling real data $\mathbf{x}$ and generated data $g_\theta(\mathbf{z})$. The two models play the following minimax game with value function $v(g_\theta, d_\phi)$:

$$\min_{g_\theta} \max_{d_\phi} v(g_\theta, d_\phi) = \mathbb{E}_{\mathbf{x}}[\log d_\phi(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - d_\phi(g_\theta(\mathbf{z})))]$$

Where $\mathbf{x}$ and $\mathbf{z}$ are sampled from the training data and the noise distribution respectively. Both models are trained simultaneously to minimize and maximize the value function respectively via gradient descent.

2.12.4 Diffusion Models

Diffusion models learn to reconstruct data that was destroyed (typically by iteratively adding noise over time). Once the model is sufficiently trained, it can be given random noise to generate entirely new samples [Sohl-Dickstein et al., 2015].

The destruction of the data is called the forward trajectory q, which adds noise to the original data $\mathbf{x}_0$ over T time-steps until it is transformed into a simple prior distribution like the Gaussian. The joint probability of the forward process at each time-step is the product of the original data distribution and the transition (added noise) probabilities at each step:

$$q(\mathbf{x_0}, \mathbf{x_1}, \mathbf{x_2}, \ldots, \mathbf{x_T}) = q(\mathbf{x}_0) \prod_{t=1}^{T} q(\mathbf{x}_t | \mathbf{x}_{t-1})$$

Sohl-Dickstein et al. [2015] used either a Gaussian diffusion kernel or a binomial diffusion kernel to model the transition $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ for their experiments.

The reconstruction of data from noise is called the reverse trajectory p, which reconstructs the original data $\mathbf{x}_0$ step-by-step from the simple prior $p(\mathbf{x}_T)$. The joint probability of the reverse process, similarly to the forward trajectory, is the product of the prior distribution and the reverse transition probabilities at each step:

$$p(\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T) = p(\mathbf{x}_T) \prod_{t=1}^{T} p(\mathbf{x}_{t-1} | \mathbf{x}_t)$$

Sohl-Dickstein et al. [2015]'s models learn to predict the mean and covariance of the kernel by minimizing the KL divergence of the forward and reverse trajectories. Modern diffusion models like Ho et al. [2020]'s DDPM predict the noise itself rather than the parameters of the noise's distribution. Ho et al. [2020] also simplifies the learning process by using mean square error (MSE) as the loss function, but the underlying theory remains the same.

3 Generative Models for PPI Prediction

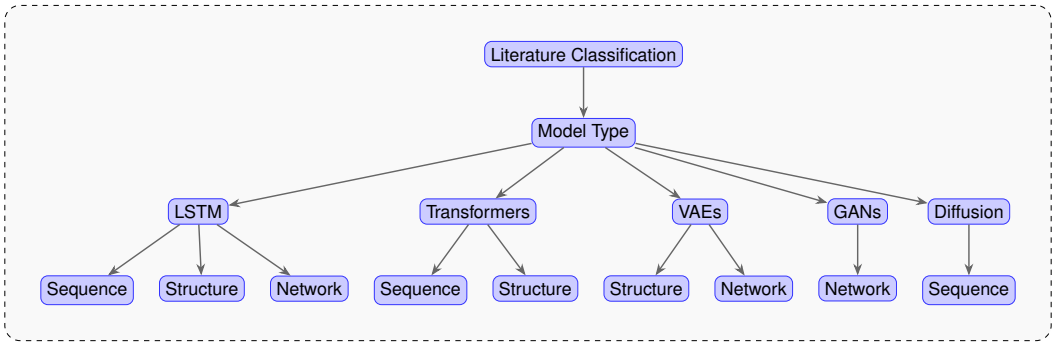

Figure 8: The structure of how PPI prediction methods are classified in this review.

Generative models are effective at learning complex patterns in large datasets. When pre-trained on a protein-related learning task, a model learns useful information about the function of each amino acid and the shape of protein structures. This information is found as the activations in the model's hidden layers, and can be extracted to obtain a rich embedding. PPI prediction techniques use these embeddings as input into a trained classifier model for the final output.

There are three sources of data to be used for predicting PPIs: primary structure, tertiary structure (which contains primary and secondary structure inherently), and interaction networks. The first uses only the amino acid sequence of the proteins, the second uses the 3D shape of the protein, which inherently contains the sequence information, and the third uses a set of known PPIs to find new ones. Many network-based approaches also use primary structure.

In this section, we classify each of the methods by the ML architecture of their model. Then, we further divide them by which protein features (sequence, structure, network) they use during training and deployment (see Figure 8). It is notable that we did not find publications for every possible classification, and it is possible that some were missed.

3.1 LSTMs

Despite being introduced over 2 decades ago, LSTMs [Hochreiter and Schmidhuber, 1997] and their derivatives are still being developed [Beck et al., 2024], and continue to be useful in the modern day. LSTM-based PLMs like UniRep [Alley et al., 2019] have recently been used to achieve high levels of accuracy in PPI prediction. UniRep [Alley et al., 2019] is a multiplicative long short-term memory (mLSTM) model trained on the UniRef50 [Suzek et al., 2014] database to generate protein sequences. As the model processes a protein, the weights of the hidden layers are extracted for each amino acid position and averaged to give the output embedding.

3.1.1 Sequence Based

With a focus on human and virus PPIs, Tsukiyama et al. [2021] proposed LSTM-PHV, a model that uses Word2Vec embeddings of amino acid k-mers as input into an LSTM, then classified using 4 fully connected layers. The model was trained on and uses sequence data.

Dong et al. [2021] proposed MTT, a model designed to identify human-virus PPIs that leverages the pre-trained UniRep. The embeddings are used as inputs for a single-layer perceptron that was fine-tuned on both human-human PPI prediction and human-virus PPI prediction to avoid overfitting on the smaller virus dataset. Multiplicative LSTMs like UniRep utilize an intermediate state $\mathbf{m}_t$, which adds some scaling to the current input using the hidden state rather than only adding them for each gate [Krause et al., 2017]. The following modifications are made to the traditional LSTM (see Section 2.12):

$$\mathbf{m}_t = (\mathbf{U}_m \mathbf{x}_t) \odot (\mathbf{W}_m \mathbf{h}_{t-1})$$
$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{x} + \mathbf{W}_i \mathbf{m}_{t-1} + \mathbf{b}_i)$$

$$\mathbf{f}_t = \sigma(\mathbf{U}_f\mathbf{x} + \mathbf{W}_f\mathbf{m}_{t-1} + \mathbf{b}_f)$$
$$\mathbf{o}_t = \sigma(\mathbf{U}_o\mathbf{x} + \mathbf{W}_o\mathbf{m}_{t-1} + \mathbf{b}_o)$$
$$\tilde{C}_t = \tanh(\mathbf{U}_c\mathbf{x}_t + \mathbf{W}_c\mathbf{m}_{t-1} + \mathbf{b}_c)$$

Sledzieski et al. [2021] proposed D-SCRIPT, a sequence based model that leverages Bepler and Berger [2021]'s bi-directional LSTM (biLSTM) to generate embeddings. These pairs of embeddings are then used to compute a 2D contact map that encodes potential physical contact sites (potential interaction sites), which is used as input for the final pooling module that outputs the interaction probability.

Bi-directional LSTMs utilize a second LSTM network that processes the input in the reverse order. The hidden state for the input x_t at time-step t is then a concatenation of both the forward network and backward network's hidden states:

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t, \overleftarrow{\mathbf{h}}_t]$$

where $\overrightarrow{\mathbf{h}}_t$ and $\overleftarrow{\mathbf{h}}_t$ are the hidden states at time-step t for the forward and backward networks respectively. This allows the hidden states to learn both the "past" and "future" contexts of the sequence.

RAPPPID was proposed by Szymborski and Emad [2022] and feeds each sequence through a two-layer bi-directional AWD-LSTM [Merity et al., 2017], and then through a fully connected layer to obtain an embedding of each protein. Finally, the embeddings are concatenated and classified by a fully connected network.

3.1.2 Network Based

Singh et al. [2022] proposed Topsy-Turvy, a model that combines D-SCRIPT [Sledzieski et al., 2021] and GLIDE [Devkota et al., 2020] to make use of both sequence and graph-based information to predict interactions. The model was designed to use graph information only during training, allowing the tool to be used with only sequence data as input.

3.1.3 Structure Based

A structure-aware extension of Topsy-Turvy, proposed by Sledzieski et al. [2023], incorporates structural data generated by Foldseek and AlphaFold 2 [Beck et al., 2024]. The embedding of the 3D structure is concatenated to the PLM embedding and trained with the same methodology as Topsy-Turvy. They aptly named the model Topsy-Turvy 3D (TT3D).

3.2 Transformers

The most common generative model based on the transformer for PPI prediction is ProtT5. ProtT5 is PLM based on Google's T5 architecture [Raffel et al., 2020]. Its rich amino acid embeddings can be used not only for PPI prediction, but also for a diverse spread of bioinformatics problems like sequence alignment [Spicer et al., 2025], protein classification [Shoombuatong et al., 2025], and a variety of prediction tasks [Schmirler et al., 2024]. Many of the recent state-of-the-art frameworks for PPI prediction make use of embeddings generated by ProtT5. Table 4 shows each of the transformer-based models reviewed in this section, the type of protein data they use, and which PLM is used to generate the embeddings.

3.2.1 Sequence Based

Dang and Vu [2024] proposed xCAPT5, a model that leverages the embeddings generated by the ProtT5-XL-UniRef50 PLM [Elnaggar et al., 2022], a multi-kernel deep convolutional siamese neural network, and the XGBoost algorithm. xCAPT5 predicts PPIs based solely on the primary structure.

xCAPT5 takes the amino acid embeddings generated by ProtT5 and feeds them through individual CNNs with varying kernel sizes. The output of each kernel size is then pooled with average and max pooling, and concatenated to make a comprehensive representation of each protein. This process allows the model to retain information about local, fine-grained features as well as abstract global features simultaneously.

Table 4: Summary of transformer-based PPI Prediction Models, including data modalities and underlying PLMs.

Model	Reference	Data	PLM Embeddings
xCAPT5	Dang and Vu [2024]	Sequence	ProtT5
MaTPIP	Ghosh and Mitra [2024]	Sequence + Manual Features	ProtT5
ESM-PEFT	Sledzieski et al. [2024]	Sequence (PEFT)	ESM-2
TUnA	Ko et al. [2024]	Sequence (Uncertainty)	ESM-2
ESM-AMP	Hosseini et al. [2025]	Sequence (Permutations)	ProtT5
PLM-Interact	Liu et al. [2025]	Sequence (Joint Enc.)	ESM-2
ppIRIS	Piochi et al. [2025]	Sequence	ESM-2 & ProstT5
AttnSeq-PPI	Sarkar and Sarkar [2026]	Sequence (Attention)	ProtT5
SpatialPPIv2	Hu and Ohue [2025]	Structure (GAT)	ProtT5
DeepGNHV	Jiang et al. [2025]	Structure (AF2)	ProtT5

The final representations of each protein are concatenated and used as input for an MLP with two densely connected layers. The internal representations of this MLP are fed into an XGBoost, which is trained to perform the final prediction. xCAPT5 attains state-of-the-art performance, but at the cost of model complexity. It also demonstrated that increasing the number of kernels can improve model performance for biological applications.

Ghosh and Mitra [2024] proposed MaTPIP (Manual and pre-Trained PLM derived feature-based Protein-protein Interaction Predictor), a sequence-based approach to PPI prediction that fuses pre-trained PLM features with manually curated sequence attributes. In particular, it fuses the ProtT5-XL-UniRef50 model's embedding with position-specific scoring matrix (PSSM), LabelEncoding, SkipGram, BLOSUM62, autocovariance (AC), pseudo amino acid composition (PSEAAC), conjoint triad (CT), and local descriptor (LD), then uses a simple MLP to perform the final binary classification.

The authors found that the features obtained from ProtT5 were the most critical component of the model. They also note that the manually curated features contribute more in the scenario where the proteins in the test set appear in the training set, but with different interacting partners. They claim that this indicates that only the PLM features are not sufficient, and complementing them with manually curated features will result in better performance.

Focusing on model efficiency, Sledzieski et al. [2024] employed parameter-efficient fine-tuning on ESM-2 and fed the embeddings through an MLP for classification. However, they still achieved state-of-the-art on the [Bernett et al., 2024] dataset, as one of the first to use it for training and testing.

Addressing the problem of model reliability, TUnA [Ko et al., 2024] uses a spectral-normalized neural Gaussian process based on Liu et al. [2020]'s paper to evaluate its uncertainty. ESM-2 protein embeddings and transformer-based encoders capture intra- and inter-protein features. Accounting for uncertainty in complex tasks can help reduce false positives and therefore the reliability of the model. Importantly, TUnA is one of the first models to be evaluated on Bernett et al. [2024]'s gold standard dataset and achieve performance better than random.

Further exploring transformer-based contextual learning, ESM-AMP [Sun et al., 2025] uses ESM-2 embeddings to encode the pairs of proteins. Then, it passes the embeddings to a transformer encoder to learn the inter- and intra-protein interaction behaviour. Finally, an MLP is used for classification.

In order to incorporate some structural information from sequence data only, C3PI [Hosseini et al., 2025] randomly shuffles segments of the input proteins into 8 random permutations to expose the model to alternative orderings of the amino acids. The intuition is that doing this will allow residues that are distant in the sequence to be close to each other as they may be in the folded 3D structure of the protein. Each of these 8 random permutations is used as input for ProtT5, whose embeddings are combined and used for final classification. C3PI is another model that made use of Bernett et al. [2024]'s gold standard dataset.

Rather than treating proteins as individual entities, PLM-Interact [Liu et al., 2025] advocates for a "joint encoding" strategy. By concatenating both sequences before they enter a fully fine-tuned

ESM-2 model, the architecture ensures that the embedding of each protein is intrinsically "aware" of the other throughout the network.

Piochi et al. [2025] proposed ppIRIS, a model that uses both ESM-2 and ProstT5 (A fine-tuned version of ProtT5) to generate its embeddings. The embeddings are manipulated by siamese fully connected networks, concatenated, and passed through one final fully connected network for classification. Interestingly, the authors evaluated the model on the [Bernett et al., 2024] gold standard, but did not report AUROC results, only accuracy, precision, recall, and f1.

The recently proposed AttnSeq-PPI [Sarkar and Sarkar, 2026] leverages ProtT5 embeddings to encode the proteins. The embeddings are then sent through a convolutional layer, and a custom self and cross attention module before being concatenated and classified by a feed-forward network.

3.2.2 Structure Based

Hu and Ohue [2025] proposed SpatialPPIv2, using both ProtT5 for sequence information embedding and graph attention networks (GATs) for 3D structural information embedding. Therefore, it uses both the primary and tertiary structure of the protein to inform its decisions.

When encoding tertiary structural data of proteins, each residue is represented as a node of a graph. The node contains information about the residue, the ProtT5 amino acid embedding in particular. An edge is added between two nodes if the distance between them in 3D space is below a certain threshold. An extra edge was added to connect each pair of residues between the two proteins to allow communication between them in the GAT. These edges are weighted according to their attention score, representing their corresponding amino acids' level of association. The learned features from the GAT are then used along with the ProtT5 embeddings, which are passed as input for a final fully connected classifier.

DeepGNHV [Jiang et al., 2025] focuses on human-virus PPI prediction and uses a similar approach to Hu and Ohue [2025]. Much like SpacialPPIv2, DeepGNHV leverages ProtT5 for the sequence embeddings, but uses the artificial structure data generated by AlphaFold 2 [Jumper et al., 2021] due to the limited data available for human-virus PPIs. The architecture is also different. The embeddings from ProtT5 were not used as input directly into the final classifier, like SpatialPPIv2, but rather were only used for the nodes of the graph.

The general format of transformer-based PPI prediction models is as follows. First, use some pre-trained PLM to embed the protein sequences, whether it be joint (concatenated before embedding) or separate. Next, use the embeddings as input for some network, usually attention-based, to learn the relationship between the proteins. Finally, use a simple classifier network to output an interaction score or probability (see Figure 9). Some models also embed the structural information of each protein. Mathematically, the interaction probability of the input pair of proteins can be defined as

$$p_{\text{interact}} = f_\theta(g_\theta(\text{concat}(\text{embeddings}, \text{structure}))$$

where

$$\text{embeddings} = \text{concat}(\text{plm}(\mathbf{a}), \text{plm}(\mathbf{b}))$$

for models that embed the sequences disjointly, or

$$\text{embeddings} = \text{plm}(\text{concat}(\mathbf{a}, \mathbf{b}))$$

for models that jointly embed the protein sequences $\mathbf{a}$ and $\mathbf{b}$ with strucural information encoded in structure (If structural information is not utilized, then this variable is empty and $\text{concat}(\text{embeddings}, \text{structure}) = \text{embeddings}$). The function $f_\theta : \mathbb{R}^{d_z \times l} \to \mathbb{R}$ is a neural network (typically transformer based), parametrized by θ that maps from the representation learning module's output space $\mathbb{R}^{d_z \times l}$ to a real number representing the interaction score or probability, and the function $g_\theta : \mathbb{R}^{d_e \times l} \to \mathbb{R}^{d_z \times l}$ is a network, parametrized by θ that maps from the PLM embeddings to some latent space $\mathbb{R}^{d_z \times l}$. Here, l is the combined length of the proteins, as PLMs like ProtT5 and ESM-2 output a vector for each amino acid in a sequence, d_z and d_e are the output dimensions of the representation learning module and the PLMs, respectively. Both f_θ and g_θ are

typically trained simultaneously, hence the common parametrization. Note that this does not imply that the networks share weights.

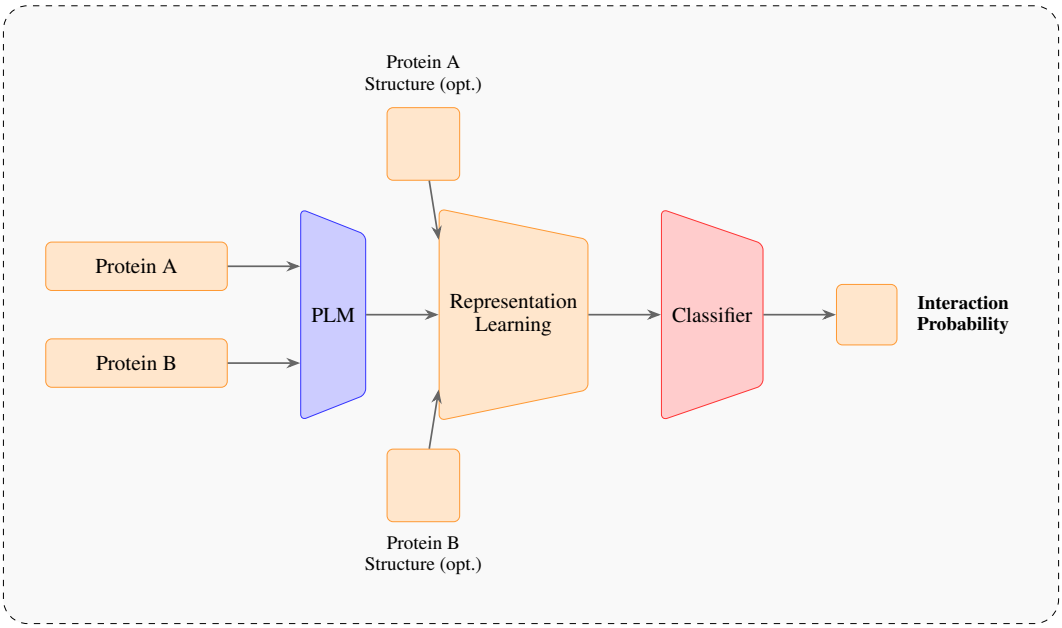

Figure 9: An illustration of the general structure of transformer-based PPI prediction models.

Some models, like xCAPT5 [Dang and Vu, 2024], keep the proteins separate until classification. Using the same notation as above, we can define this approach as:

$$p_{\text{interact}} = f_\theta(\text{concat}(g_\theta(\text{plm}(\mathbf{a}), \text{plm}(\mathbf{b}))))$$

The functions f_θ and g_θ can be any form of NN. For example, in xCAPT5 [Dang and Vu, 2024], f_θ is a MLP followed by XGBoost, and g_θ is a series of CNNs. In TUnA [Ko et al., 2024], f_θ is a spectral-normalized Gaussian process, and g_θ is a custom transformer based module. In C3PI [Hosseini et al., 2025], f_θ is a MLP and g_θ is their custom "puzzler" and "entangler" modules etc.

For structure-based methods, the data is encoded separately from the sequences and concatenated to the transformer embeddings. SpatialPPIv2 [Hu and Ohue, 2025] uses a GAT to encode the structural information of each protein and concatenates them all together prior to representation learning and classification.

3.3 Variational Autoencoders

VAEs process information in a single pass, unlike RNNs, LSTMs, or transformers. This means that they must learn to represent the biological mechanisms and details of every amino acid in the protein, as well as their interactions with each other, all at once, and compress them into the latent space. This leads to serious loss of detailed information, and a focus on the "big picture". In sequence and structure-based PPI prediction, individual residue-level details are critical. Methods using VAEs are used as a complement to other architectures, or use interaction network information for their predictions. Table 5 provides a brief summary of VAE-based methods for PPI prediction.

3.3.1 Structure Based

Wu et al. [2024] proposed MAPE-PPI, a model that uses a variant of a vector quantized variational autoencoder (VQ-VAE) to create a "microenvironment codebook". Using this codebook as a tool to encode proteins, a fully connected classifier network predicts the interaction type. MAPE-PPI uses the primary structure, and AlphaFold 2 predicted tertiary structure to predict interactions.

Table 5: Summary of VAE-based methods for PPI prediction.

Data	Model	Reference	Highlights
Sequence & Structure	MAPE-PPI	Wu et al. [2024]	Uses a VQ-VAE variant with a "microenvironment codebook" to encode primary and AlphaFold 2 tertiary structures. Trained via Masked Codebook Modeling.
Sequence & Structure	VQ-VAE	Kiouri et al. [2025b]	Hybrid model fusing pre-trained VQ modules (from MAPE-PPI) with a bi-directional attention mechanism. Tailored for the human gut microbiome.
Sequence & Network	S-VGAE	Yang et al. [2020]	Signed Variational Graph Auto Encoder using GCNs. Combines CT-extracted sequence features with local network topology to reconstruct the adjacency matrix.
Network	HPIP	Xiao et al. [2021]	Partitions sparse PPIN matrices into blocks to train multiple VAEs. Sums output probabilities to predict high-confidence interactions in incomplete networks.

MAPE-PPI's microenvironment codebook was trained using masked codebook modeling (MCM) and by treating it as a learnable variable during the VQ-VAE training (no masking). Masked codebook modeling was inspired by masked language modeling from NLP, where certain tokens from the input are "masked" or "corrupted", and the model is tasked with predicting the missing tokens. The authors propose masking the codebook vectors directly rather than the input vectors and reconstructing the inputs based on the remaining codes. The intention of this is to avoid the possibility that random masking can lead to unwanted side effects [Shi et al., 2022, Li et al., 2022]. They linearly combine the loss from the unmasked task with the MCM task to be the total loss function used for training. MAPE-PPI outperformed other methods like PIPR [Chen et al., 2019] and DPPI [Hashemifar et al., 2018] on binary PPI prediction tests available at the time and provided a new representation learning model that generates high-quality protein embeddings.

Training their model specifically for predicting PPIs in the human gut microbiome, Kiouri et al. [2025b] proposed an architecture that uses concepts from VAEs and transformers. In particular, the pre-trained encoder and VQ modules from [Wu et al., 2024] are used to generate embeddings of each protein. These embeddings are then fused into a unified pair representation by a bi-directional attention module that uses an attention mechanism not dissimilar to the encoder of the original transformer, then concatenated and projected to obtain the final embedding. The final classification module is a series of fully connected layers that output a scalar interaction score.

3.3.2 Network Based

Yang et al. [2020] proposed a framework for PPI prediction that incorporates sequence and interaction network information, leveraging their proposed variation of a VAE. The framework feeds the sequence information extracted by CT, and the local neighbourhood of the interaction network into a signed variational graph autoencoder (S-VGAE). The embeddings are extracted, and classified using a simple neural network.

The proposed S-VGAE uses graph convolutional networks (GCN) as the encoder.

$$q(\mathbf{z}|\mathbf{x}, \mathbf{A}) = \prod_{\mathbf{z}} \mathcal{N}(\mathbf{z}|\mu, \text{diag}(\sigma^2))$$

parametrized by GCNs: $\mu = \text{GCN}_\mu(\mathbf{x}, \mathbf{A})$ and $\log \sigma = \text{GCN}_\sigma(\mathbf{x}, \mathbf{A})$. The decoder tries to generate the adjacency matrix $\mathbf{A}$ from the interaction network by assigning the logistic-sigmoid σ of the inner product of each pair of embeddings $\mathbf{z}_i$ and $\mathbf{z}_j$ as the interaction probability:

$$p(\mathbf{A}|\mathbf{z}) = \prod_{i=1} \prod_{j=1} p(\mathbf{A}_{ij}|\mathbf{z}_i, \mathbf{z}_j)$$

where

$$p(\mathbf{A}_{ij} = 1|\mathbf{z}_i, \mathbf{z}_j) = \sigma(\mathbf{z}_i^T \mathbf{z}_j)$$

HPIP [Xiao et al., 2021] is a VAE prediction model that predicts PPIs with a high level of confidence based on sparse (incomplete) protein-protein interaction networks (PPINs). HPIP partitions the matrix representation of the PPIN column-wise into 4 blocks, then generates 4 input matrices for 4 VAEs by selecting 3 of the partitions. Finally, summing the output probability matrices of each of the VAE-based predictors. The interactions are filtered to exclude known PPIs, and the remaining interactions with the highest probability sum are considered predicted, high-confidence interactions. HPIP fills out sparse PPINs with high-confidence predicted interactions. The authors argue that because the PPIN matrix is symmetric (PPINs are undirected graphs), blocking won't cause any information loss for each protein. They also argue that by parallelizing the workflow with multiple models, their architecture has potential for scaling.

3.4 Generative Adversarial Networks

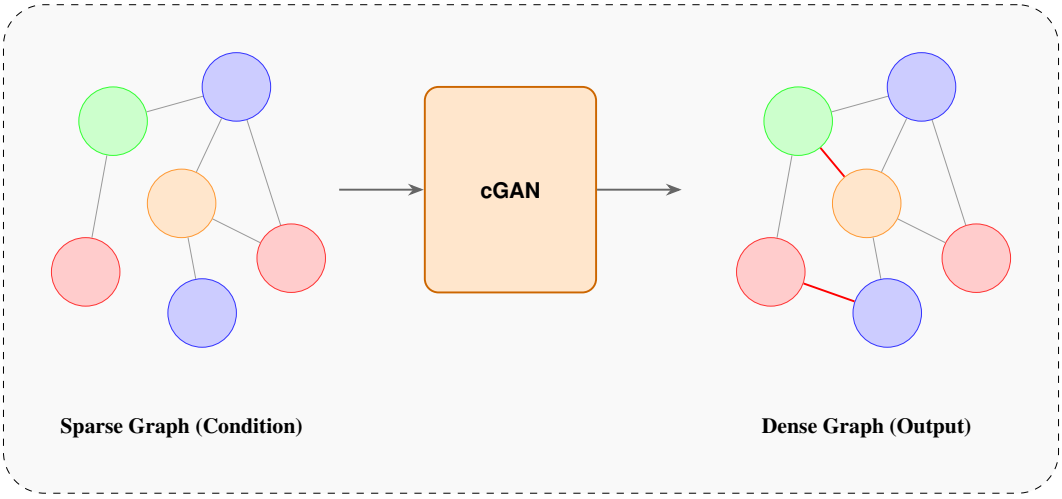

Figure 10: Balogh et al. [2022]'s cGAN PPI link prediction model at inference. The incomplete graph is passed as the condition to the cGAN generator, which completes the graph.

While GANs are effective at generating new examples of data, they do not explicitly learn the underlying distribution. This means that they are not required to learn a latent representation for proteins, and therefore cannot be used to generate embeddings. Without the ability to create embeddings of protein sequences or structures, they cannot be used for sequence or structure-based PPI prediction. However, this is not a problem for network-based methods since a GAN can be trained to generate an entire PPI network.

3.4.1 Network Based

Balogh et al. [2022] proposed a PPI network-based method for PPI prediction that uses a conditional generative adversarial network. The model is given lower connectivity subgraphs and trained to generate higher connectivity subgraphs (see Figure 10). This gives the model the ability to predict new links when testing on more complete networks. Most notably, it does not require any negative examples, as it does not directly classify individual pairs of proteins, but rather outputs an interaction score for every pair at once by generating a new, more connected adjacency matrix. Due to the clearly insufficient datasets of negative examples, avoiding the need for them is important.

The alternative is supplementing the negative datasets with artificial negative samples from random pairs of proteins, which trains the classifier to learn the distinction between positive examples and unlabeled examples. Moreover, information about the proteins themselves was not used, only the interaction network, eliminating the dependence on external datasets.

The model is based on the Pix2Pix architecture [Isola et al., 2017], modifying the discriminator to use a CNN in place of PatchGAN. The discriminator uses Wasserstein distance-based loss, causing it to act as a judge of the realism of the generator's output rather than classifying it as real or fake. This avoids cases of extreme imbalance where the discriminator learns at a much quicker rate than the generator, causing the gradient to vanish, or, on the other end, mode collapse. The authors note that their architecture is but one of many variations of the GAN model family, and that future research should investigate different architectures for both the generator and discriminator.

The interaction network is stored as an $n \times n$ adjacency matrix $\mathbf{M}^{n \times n}$, where n is the number of proteins in the network. During training, $\mathbf{M}$ is thinned (some elements in $\mathbf{M}$ are set to 0), and is passed to the cGAN generator to fill it back in. In this case, the discriminator tries to maximize the difference between its realism score for real pairs $(\mathbf{M}, \mathbf{Y})$ and the generator (g_θ)'s pairs $(\mathbf{M}, g_\theta(\mathbf{M}, \mathbf{Z}))$. The generator's goal is to fool the discriminator and match the ground truth as closely as possible. This minimax game can be defined as:

$$\min_{g_\theta} \max_{d_\phi} \left(\mathbb{E}_{\mathbf{M},\mathbf{Y}}[d_\phi(\mathbf{M}, \mathbf{Y})] - \mathbb{E}_{\mathbf{M},\mathbf{Z}}[d_\phi(\mathbf{M}, g_\theta(\mathbf{M}, \mathbf{Z}))]\right) + \lambda \mathbb{E}_{\mathbf{M},\mathbf{Y},\mathbf{Z}}[\|\mathbf{Y} - g_\theta(\mathbf{M}, \mathbf{Z})\|_1]$$

where $\mathbf{M}$ is the condition (thinned adjacency matrix), $\mathbf{Y}$ is the original adjacency matrix (ground truth), $\mathbf{Z}$ is is the noise matrix, and λ is a hyperparameter for adjusting the weight of the reconstruction loss $\mathbb{E}_{\mathbf{M},\mathbf{Y},\mathbf{Z}}[\|\mathbf{Y} - G(\mathbf{M}, \mathbf{Z})\|_1]$.

3.5 Diffusion Models

Diffusion models recently started gaining popularity with the release of image generation models like DALL-E [Ramesh et al., 2021], and more recently, text-to-video models like Sora [Brooks et al., 2024]. Diffusion models have started to be applied to the PPI prediction problem, and other problems in bioinformatics like protein structure prediction [Abramson et al., 2024].

3.5.1 Sequence Based

Wang et al. [2024] proposed a diffusion protein language model (DPLM). DPLM is a discrete diffusion model based on recent work on diffusion like Sohl-Dickstein et al. [2015], Hoogeboom et al. [2021], and Austin et al. [2023]. A discrete diffusion model uses a categorical (discrete) distribution to model the transition kernel $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ rather than a Gaussian or binomial continuous distribution.In particular, Wang et al. [2024] define the forward transition kernel as

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = Cat(\mathbf{x}_t; \beta_t \mathbf{x}_{t-1} + (1 - \beta_t)\mathbf{q}_{noise})$$

where $\mathrm{Cat}(\mathbf{x}; \mathbf{p})$ is a categorical distribution on $\mathbf{x}$ parameterized by a vector $\mathbf{p}$, $\mathbf{q}_{\text{noise}}$ is the probability vector of the distribution $\mathrm{Cat}(\mathbf{x}_t; \mathbf{q}_{\text{noise}})$ (stationary distribution), and $0 < \beta_t < 1$ is a weight variable scheduling how much noise is added at each timestep. The reverse process is parameterized by a distribution that combines $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \bar{\mathbf{x}}_0)$ with a distribution whose logits are predicted by a neural network $\bar{p}_\theta(\bar{\mathbf{x}}_0|\mathbf{x}_t)$ summed over the one-hot representations of $\mathbf{x}_0$, $\bar{\mathbf{x}}_0$.

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \sum_{\bar{\mathbf{x}}_0} q(\mathbf{x}_{t-1}, \mathbf{x}_t|\bar{\mathbf{x}}_0)\bar{p}_\theta(\bar{\mathbf{x}}_0|\mathbf{x}_t)$$

DPLM was tested on a multitude of both generative and discriminative downstream tasks, including PPI prediction. It outperformed ESM2 [Lin et al., 2023], a popular PLM used for PPI prediction, on all tasks. Wang et al. [2024] demonstrated that diffusion models for bioinformatics require further investigation.

Quinn et al. [2025] proposed the use of a denoising diffusion model operating on a protein autoencoder's latent space. The models' embeddings were evaluated on a variety of discriminative tasks, including PPI prediction, but did not perform at the level of masked language modeling or previous work on diffusion approaches [Wang et al., 2024].

Table 6: Model architecture and AUPR on Bernett et al. [2024]'s gold standard PPI dataset. "*" denotes results from Bernett et al. [2024], otherwise results are from their paper of origin. "—" denotes models that have not been tested on the gold standard dataset.

Model Type	Model	Reference	AUROC
RNN	*PIPR	Chen et al. [2019]	0.532
LSTM	LSTM-PHV	Tsukiyama et al. [2021]	—
	MTT	Dong et al. [2021]	—
	*D-SCRIPT	Sledzieski et al. [2021]	0.499
	RAPPPID	Szymborski and Emad [2022]	—
	*Topsy-Turvy	Singh et al. [2022]	0.587
	TT3D	Sledzieski et al. [2023]	—
Transformer	xCAPT5	Dang and Vu [2024]	—
	MaTPIP	Ghosh and Mitra [2024]	—
	D-SCRIPT	Sledzieski et al. [2024]	0.690
	ESM_AMP	Sun et al. [2025]	0.690
	TUnA	Ko et al. [2024]	0.700
	C3PI	Hosseini et al. [2025]	0.703
	PLM-Interact	Liu et al. [2025]	0.700
	ppIRIS	Piochi et al. [2025]	—
	AttnSeq-PPI	Sarkar and Sarkar [2026]	0.705
	SpacialPPIv2	Hu and Ohue [2025]	—
	DeepGNVH	Jiang et al. [2025]	—
VAE	MAPE-PPI	Wu et al. [2024]	—
	Structure-based Gut	Kiouri et al. [2025b]	—
	Graph-based	Yang et al. [2020]	—
	HPIP	Xiao et al. [2021]	—
GAN	Efficient GAN	Balogh et al. [2022]	—
Diffusion	DPLM	Wang et al. [2024]	—
	Denoising Diffusion	Quinn et al. [2025]	—

4 Discussion

Jiang et al. [2025] showed that using predicted structures from algorithms like AlphaFold 2 can be useful for enhancing smaller datasets where tertiary structure data is not available. Abramson et al. [2024] introduced AlphaFold 3, a new iteration of AlphaFold that uses a transformer-based denoising (diffusion) model to generate accurate structures corresponding to sequences. Applying this latest innovation in protein structure prediction could boost performance and allow for high-quality extensions for smaller datasets.

While Quinn et al. [2025] and Wang et al. [2024] were able to apply diffusion models to the problem of PPI prediction, there is still potential for future research in this area. A more sophisticated and rigorous fine-tuning approach on each of the classification tasks outlined in their papers, including PPI prediction, should be explored further.

Similarly, there has been little work applying adversarial networks to PPI prediction. Balogh et al. [2022] demonstrated that cGANs can be effectively used for this problem by leveraging interaction network information. Future research could investigate different approaches using primary or secondary structural information alongside the network information or explore different GAN variants.

On a more general note, models that train a separate classifier module must choose between having a limited number of true negative examples or artificially generating them. Both of these have drawbacks and are current limitations of the field. One solution to this is to avoid the need for negative examples during training at all, Xiao et al. [2021] and Balogh et al. [2022] employ this strategy. A method that leverages the power of generative AI to generate adjacency matrices to avoid the need for negative examples, while also making use of the rich information contained in

the proteins themselves, is an interesting potential research avenue. Though the applicability of these models is questionable, since in the case that there is already an interaction graph for the interactome of interest, there is no reason to predict interactions. Models that memorize interaction network topology have been shown to have poor generalizability [Bernett et al., 2024].

Another general limitation is the lack of use of a high-quality, standardized test set. While Bernett et al. [2024] created such a dataset, it was released in 2024 and therefore has not yet been widely adopted (see Table 6). Most models have not been trained or evaluated on the new gold standard dataset. This means that some models that performed well may have success on the harder dataset, given that they are trained on it. Some future research could be to retrain the outdated models, as their poor performance could possibly be attributed to low-quality training data.

Of the models that were trained on Bernett et al. [2024]'s gold standard (Sledzieski et al. [2024], TUnA, C3PI, and PLM-Interact, etc., see Table 6), they perform similarly, but there is much room for improvement. It is apparent that the problem of PPI prediction is harder than initially thought, given the high and sometimes near-perfect performance reported on previous datasets. For all future research in this field, the Bernett et al. [2024] gold standard should be at least one of the datasets being used.

5 Conclusion

Recent advances in generative models have advanced the field of computational PPI prediction methods. In particular, they are capable of learning the complex underlying mechanisms of proteins, which can be leveraged to predict interactions at a high level of accuracy, though Bernett et al. [2024] demonstrated that most of these claimed accuracies are reliant on sequence similarity and node degree information.

Many recent models combine primary and tertiary structural data, or interaction network information, to enhance performance [Hu and Ohue, 2025, Jiang et al., 2025, Wu et al., 2024, Kiouri et al., 2025b]. Some have even demonstrated that using artificially generated structures can still boost performance, especially on smaller datasets [Wu et al., 2024, Jiang et al., 2025].

In conclusion, generative models have advanced the state-of-the-art in PPI prediction, but remain plagued by inconsistent testing and unsatisfactory data. With the new Bernett dataset, this limitation may soon be overcome and could result in a faster, cheaper drug discovery process.

References

J. Abramson, J. Adler, J. Dunger, R. Evans, T. Green, A. Pritzel, O. Ronneberger, L. Willmore, A. J. Ballard, J. Bambrick, S. W. Bodenstein, D. A. Evans, C.-C. Hung, M. O'Neill, D. Reiman, K. Tunyasuvunakool, Z. Wu, A. Žemgulytė, E. Arvaniti, C. Beattie, O. Bertolli, A. Bridgland, A. Cherepanov, M. Congreve, A. I. Cowen-Rivers, A. Cowie, M. Figurnov, F. B. Fuchs, H. Gladman, R. Jain, Y. A. Khan, C. M. R. Low, K. Perlin, A. Potapenko, P. Savy, S. Singh, A. Stecula, A. Thillaisundaram, C. Tong, S. Yakneen, E. D. Zhong, M. Zielinski, A. Žídek, V. Bapst, P. Kohli, M. Jaderberg, D. Hassabis, and J. M. Jumper. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 2024.

S. Akbarzadeh, O. Coşkun, and B. Günçer. Studying protein-protein interactions: Latest and most popular approaches. *Journal of Structural Biology*, 216(4), 2024. ISSN 1047-8477.

B. Alberts, A. Johnson, J. Lewis, M. Raff, K. Roberts, and P. Walter. The Shape and Structure of Proteins. In *Molecular Biology of the Cell. 4th edition*. Garland Science, 2002.

E. C. Alley, G. Khimulya, S. Biswas, M. AlQuraishi, and G. M. Church. Unified rational protein engineering with sequence-based deep representation learning. *Nature Methods*, 2019.

S. F. Altschul, T. L. Madden, A. A. Schäffer, J. Zhang, Z. Zhang, W. Miller, and D. J. Lipman. Gapped BLAST and PSI-BLAST: a new generation of protein database search programs. *Nucleic Acids Research*, 1997.

R. Archana and P. S. E. Jeevaraj. Deep learning models for digital image processing: a review. *Artificial Intelligence Review*, 2024.

E. Asgari and M. R. K. Mofrad. Continuous Distributed Representation of Biological Sequences for Deep Proteomics and Genomics. *PLoS ONE*, 2015.

T. T. Aumentado-Armstrong, B. Istrate, and R. A. Murgita. Algorithmic approaches to protein-protein interaction site prediction. *Algorithms for Molecular Biology*, 10(1), 2015. ISSN 1748-7188.

J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models in discrete state-spaces, 2023.

O. M. Balogh, B. Benczik, A. Horváth, M. Pétervári, P. Csermely, P. Ferdinandy, and B. Ágg. Efficient link prediction in the protein-protein interaction network using topological information in a generative adversarial network machine learning model. *BMC bioinformatics*, 2022.

D. Bank, N. Koenigstein, and R. Giryes. Autoencoders. In L. Rokach, O. Maimon, and E. Shmueli, editors, *Machine Learning for Data Science Handbook: Data Mining and Knowledge Discovery Handbook*, pages 353–374. Springer International Publishing, 2023.

M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter. xLSTM: Extended Long Short-Term Memory. In *NeurIPS*, 2024.

A. Ben-Hur and W. S. Noble. Choosing negative examples for the prediction of protein-protein interactions. *BMC Bioinformatics*, 2006.

T. Bepler and B. Berger. Learning the Protein Language: Evolution, Structure and Function. *Cell systems*, 2021.

H. M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T. N. Bhat, H. Weissig, I. N. Shindyalov, and P. E. Bourne. The Protein Data Bank. *Nucleic Acids Research*, 2000.

J. Bernett, D. B. Blumenthal, and M. List. Cracking the black box of deep sequence-based protein–protein interaction prediction. *Briefings in Bioinformatics*, 25, 2024. ISSN 1477-4054.

D. Bertoni, M. Tsenkov, P. Magana, S. Nair, I. Pidruchna, M. Querino Lima Afonso, A. Midlik, U. Paramval, D. Lawal, A. Tanweer, M. Last, R. Patel, A. Laydon, D. Lasecki, N. Dietrich, H. Tomlinson, A. Žídek, T. Green, O. Kovalevskiy, A. Lau, S. Kandathil, N. Bordin, I. Sillitoe, M. Mirdita, D. Jones, C. Orengo, M. Steinegger, J. R. Fleming, and S. Velankar. AlphaFold Protein Structure Database 2025: a redesigned interface and updated structural coverage. *Nucleic Acids Research*, 2025.

P. Blohm, G. Frishman, P. Smialowski, F. Goebels, B. Wachinger, A. Ruepp, and D. Frishman. Negatome 2.0: a database of non-interacting proteins derived by literature mining, manual annotation and protein structure analysis. *Nucleic Acids Research*, 2013.

P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 2016.

N. Brandes, D. Ofer, Y. Peleg, N. Rappoport, and M. Linial. ProteinBERT: a universal deep-learning model of protein sequence and function. *Bioinformatics*, 38(8), 2022. ISSN 1367-4803.

T. Brooks, B. Peebles, C. Holmes, W. DePue, Y. Guo, L. Jing, D. Schnurr, J. Taylor, T. Luhman, E. Luhman, C. Ng, R. Wang, and A. Ramesh. Video generation models as world simulators. 2024.

T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language Models are Few-Shot Learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

A. Brückner, C. Polge, N. Lentze, D. Auerbach, and U. Schlattner. Yeast Two-Hybrid, a Powerful Tool for Systems Biology. *International Journal of Molecular Sciences*, 10(6):2763–2788, 2009. ISSN 1422-0067.

A. Calderone, L. Castagnoli, and G. Cesareni. mentha: a resource for browsing integrated protein-interaction networks. *Nature Methods*, 2013.

F. Charih, J. R. Green, and K. K. Biggar. Sequence-Based Protein–Protein Interaction Prediction and Its Applications in Drug Discovery. *Cells*, 2025.

M. Chen, C. J. T. Ju, G. Zhou, X. Chen, T. Zhang, K.-W. Chang, C. Zaniolo, and W. Wang. Multi-faceted protein–protein interaction prediction based on Siamese residual RCNN. *Bioinformatics*, 35(14), 2019.

K.-C. Chou. Prediction of protein cellular attributes using pseudo-amino acid composition. *Proteins: Structure, Function, and Bioinformatics*, 2001.

T. U. Consortium. UniProt: the Universal Protein Knowledgebase in 2025. *Nucleic Acids Research*, 2024.

C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 1995.

T. Dandekar, B. Snel, M. Huynen, and P. Bork. Conservation of gene order: a fingerprint of proteins that physically interact. *Trends in Biochemical Sciences*, 1998.

T. H. Dang and T. A. Vu. xCAPT5: protein-protein interaction prediction using deep and wide multi-kernel pooling convolutional neural networks with protein language model. *BMC bioinformatics*, 2024.

K. Devkota, J. M. Murphy, and L. J. Cowen. GLIDE: combining local methods and diffusion state embeddings to predict missing interactions in biological networks. *Bioinformatics (Oxford, England)*, 2020.

J. Devlin, M.-W. Chang, K. Lee, and K. N. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. 2018.

Z. Ding and D. Kihara. Computational Methods for Predicting Protein-Protein Interactions Using Various Protein Features. *Current Protocols in Protein Science*, 2018.

T. N. Dong, G. Brogden, G. Gerold, and M. Khosla. A multitask transfer learning framework for the prediction of virus-human protein–protein interactions. *BMC Bioinformatics*, 2021.

J. L. Elman. Finding Structure in Time. *Cognitive Science*, 14(2), 1990. ISSN 1551-6709.

A. Elnaggar, M. Heinzinger, C. Dallago, G. Rehawi, Y. Wang, L. Jones, T. Gibbs, T. Feher, C. Angerer, M. Steinegger, D. Bhowmik, and B. Rost. ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

S. Ghosh and P. Mitra. MaTPIP: A deep-learning architecture with eXplainable AI for sequence-driven, feature mixed protein-protein interaction prediction. *Computer Methods and Programs in Biomedicine*, 2024.

T. Goldfarb, V. K. Kodali, S. Pujar, V. Brover, B. Robbertse, C. M. Farrell, D.-H. Oh, A. Astashyn, O. Ermolaeva, D. Haddad, W. Hlavina, J. Hoffman, J. D. Jackson, V. S. Joardar, D. Kristensen, P. Masterson, K. M. McGarvey, R. McVeigh, E. Mozes, M. R. Murphy, S. S. Schafer, A. Souvorov, B. Spurrier, P. K. Strope, H. Sun, A. R. Vatsan, C. Wallin, D. Webb, J. R. Brister, E. Hatcher, A. Kimchi, W. Klimke, A. Marchler-Bauer, K. D. Pruitt, F. Thibaud-Nissen, and T. D. Murphy. NCBI RefSeq: reference sequence standards through 25 years of curation and annotation. *Nucleic Acids Research*, 2024.

I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 2014.

Y. Guo, L. Yu, Z. Wen, and M. Li. Using support vector machine combined with auto covariance to predict protein-protein interactions from protein sequences. *Nucleic Acids Research*, 2008.

Z. Guo, J. Liu, Y. Wang, M. Chen, D. Wang, D. Xu, and J. Cheng. Diffusion models in bioinformatics and computational biology. *Nature reviews bioengineering*, 2024.

S. Hashemifar, B. Neyshabur, A. A. Khan, and J. Xu. Predicting protein–protein interactions through sequence-based deep learning. *Bioinformatics*, 2018.

J. Ho, A. Jain, and P. Abbeel. Denoising Diffusion Probabilistic Models. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *NeurIPS*. Curran Associates, Inc., 2020.

S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9:1735–1780, 11 1997.

E. Hoogeboom, D. Nielsen, P. Jaini, P. Forré, and M. Welling. Argmax flows and multinomial diffusion: Learning categorical distributions, 2021.

S. Hosseini, G. B. Golding, and L. Ilie. C3PI: Component Puzzle Protein-Protein Interaction Prediction. 2025.

W. Hu and M. Ohue. SpatialPPIv2: Enhancing protein–protein interaction prediction through graph neural networks with protein language models. *Computational and Structural Biotechnology Journal*, 2025.

P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. Image-to-image translation with conditional adversarial networks. In *IEEE*, 2017.

L. Jiang, X. Yang, X. Guo, D. Li, J. Li, S. Wuchty, W. Shi, and Z. Zhang. Graph neural network integrated with pretrained protein language model for predicting human–virus protein–protein interactions. *Briefings in Bioinformatics*, 2025.

J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 2021.

D. P. Kingma and M. Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

D. P. Kiouri, G. C. Batsis, and C. T. Chasapis. Structure-Based Approaches for Protein–Protein Interaction Prediction Using Machine Learning and Deep Learning. *Biomolecules*, 2025a.

D. P. Kiouri, G. C. Batsis, and C. T. Chasapis. Structure-Based Deep Learning Framework for Modeling Human –Gut Bacterial Protein Interactions. *Proteomes*, 2025b.

Y. S. Ko, J. Parkinson, C. Liu, and W. Wang. TUnA: an uncertainty-aware transformer model for sequence-based protein–protein interaction prediction. *Briefings in Bioinformatics*, 2024.

B. Krause, L. Lu, I. Murray, and S. Renals. Multiplicative lstm for sequence modelling, 2017.

A. LaPelusa and R. Kaushik. Physiology, Proteins. In *StatPearls*. StatPearls Publishing, Treasure Island (FL), 2022.

Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation*, 1(4), 1989. ISSN 0899-7667.

A. Lehninger. *Biochemistry: The Molecular Basis of Cell Structure and Function*. Worth Publishers, 1970.

G. Li, H. Zheng, D. Liu, C. Wang, B. Su, and C. Zheng. SemMAE: Semantic-Guided Masking for Learning Masked Autoencoders. In *NeurIPS*, 2022.

L. Licata, L. Briganti, D. Peluso, L. Perfetto, M. Iannuccelli, E. Galeota, F. Sacco, A. Palma, A. P. Nardozza, E. Santonico, L. Castagnoli, and G. Cesareni. MINT, the molecular interaction database: 2012 update. *Nucleic Acids Research*, 2011.

Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smetanin, R. Verkuil, O. Kabeli, Y. Shmueli, A. dos Santos Costa, M. Fazel-Zarandi, T. Sercu, S. Candido, and A. Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 2023.

D. Liu, F. Young, K. D. Lamb, A. Claudio Quiros, A. Pancheva, C. J. Miller, C. Macdonald, D. L. Robertson, and K. Yuan. PLM-interact: extending protein language models to predict protein-protein interactions. *Nature Communications*, 2025.

J. Liu, Z. Lin, S. Padhy, D. Tran, T. Bedrax Weiss, and B. Lakshminarayanan. Simple and Principled Uncertainty Estimation with Deterministic Deep Learning via Distance Awareness. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 7498–7512. Curran Associates, Inc., 2020.

S. Merity, N. S. Keskar, and R. Socher. Regularizing and optimizing lstm language models, 2017.

T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. 2013.

R. Morris, K. A. Black, and E. J. Stollar. Uncovering protein function: from classification to complexes. *Essays in Biochemistry*, 2022.

H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian. A comprehensive overview of large language models. *ACM Trans. Intell. Syst. Technol.*, 16(5), 2025. ISSN 2157-6904.

S. Orchard, M. Ammari, B. Aranda, L. Breuza, L. Briganti, F. Broackes-Carter, N. H. Campbell, G. Chavali, C. Chen, N. del Toro, M. Duesbury, M. Dumousseau, E. Galeota, U. Hinz, M. Iannuccelli, S. Jagannathan, R. Jimenez, J. Khadake, A. Lagreid, L. Licata, R. C. Lovering, B. Meldal, A. N. Melidoni, M. Milagros, D. Peluso, L. Perfetto, P. Porras, A. Raghunath, S. Ricard-Blum, B. Roechert, A. Stutz, M. Tognolli, K. van Roey, G. Cesareni, and H. Hermjakob. The MIntAct project—IntAct as a common curation platform for 11 molecular interaction databases. *Nucleic Acids Research*, 2013.

R. Oughtred, J. Rust, C. Chang, B.-J. Breitkreutz, C. Stark, A. Willems, L. Boucher, G. Leung, N. Kolas, F. Zhang, S. Dolma, J. Coulombe-Huntington, A. Chatr-aryamontri, K. Dolinski, and M. Tyers. The BioGRID database: A comprehensive biomedical resource of curated protein, genetic, and chemical interactions. *Protein Science*, 2021.

L. F. Piochi, D. Tang, J. Malmström, Y. Karami, and H. Khakzad. ppIRIS: deep learning for proteome-wide prediction of bacterial protein-protein interactions. *bioRxiv*, 2025.

L. Qin, Q. Chen, X. Feng, Y. Wu, Y. Zhang, Y. Li, M. Li, W. Che, and P. S. Yu. Large language models meet nlp: A survey, 2025.

E. Quinn, G. Jebali, M. Seince, and O. Bent. Discriminative protein sequence modelling with latent space diffusion, 2025.

A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving Language Understanding by Generative Pre-Training. 2018.

C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 2020.

A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever. Zero-shot text-to-image generation, 2021.

V. S. Rao, K. Srinivas, G. N. Sujini, and G. N. S. Kumar. Protein-Protein Interaction Detection: Methods and Analysis. *International Journal of Proteomics*, 2014.

J. Reece, L. Urry, M. Cain, S. Wasserman, P. Minorsky, and R. Jackson. *Campbell Biology*. Pearson Education, 2011.

F. Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. ISSN 0033-295X.

D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088), 1986. ISSN 1476-4687.

D. Sarkar and C. Sarkar. AttnSeq-PPI: Enhancing protein-protein interaction network prediction using transfer learning-driven hybrid attention. *Biochimica et Biophysica Acta (BBA) - Proteins and Proteomics*, 1874(1), 2026. ISSN 1570-9639.

R. Schmirler, M. Heinzinger, and B. Rost. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 2024.

J. Shen, J. Zhang, X. Luo, W. Zhu, K. Yu, K. Chen, Y. Li, and H. Jiang. Predicting protein–protein interactions based only on sequences information. *Proceedings of the National Academy of Sciences of the United States of America*, 2007.

Y. Shi, N. Siddharth, P. Torr, and A. R. Kosiorek. Adversarial masking for self-supervised learning. In *ICML*, 2022.

W. Shoombuatong, N. Schaduangrat, P. Mookdarsanit, J. Nikom, and L. Mookdarsanit. Advancing the accuracy of clathrin protein prediction through multi-source protein language models. *Scientific Reports*, 2025.

R. Singh, K. Devkota, S. Sledzieski, B. Berger, and L. Cowen. Topsy-Turvy: integrating a global view into sequence-based PPI prediction. *Bioinformatics*, 2022.

S. Sivakumar, L. S. Videla, T. Rajesh Kumar, J. Nagaraj, S. Itnal, and D. Haritha. Review on Word2Vec Word Embedding Neural Net. In *2020 International Conference on Smart Electronics and Communication (ICOSEC)*, pages 282–290, Sept. 2020.

S. Sledzieski, R. Singh, L. Cowen, and B. Berger. D-SCRIPT translates genome to phenome with sequence-based, structure-aware, genome-scale predictions of protein-protein interactions. *Cell systems*, 2021.

S. Sledzieski, K. Devkota, R. Singh, L. Cowen, and B. Berger. TT3D: Leveraging precomputed protein 3D sequence models to predict protein–protein interactions. *Bioinformatics*, 2023.

S. Sledzieski, M. Kshirsagar, M. Baek, R. Dodhia, J. Lavista Ferres, and B. Berger. Democratizing protein language models with parameter-efficient fine-tuning. *Proceedings of the National Academy of Sciences*, 121(26), 2024.

J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli. Deep Unsupervised Learning using Nonequilibrium Thermodynamics . In *ICML*, 2015.

R. Spicer, N. Raychawdhary, S. Danwada, P. Udomprasert, C. Seals, and S. Bhattacharya. Evaluating the significance of embedding-based protein sequence alignment with clustering and double dynamic programming for remote homology. *Scientific Reports*, 2025.

J. Stephenson and K. R. Karnati. Recent trends in machine learning and deep learning-based prediction of G-protein coupled receptor-ligand binding affinities. *Frontiers in Bioinformatics*, 5, 2025. ISSN 2673-7647.

T. Sun, B. Zhou, L. Lai, and J. Pei. Sequence-based prediction of protein protein interaction using a deep-learning algorithm. *BMC Bioinformatics*, 2017.

Y. Sun, R. Wang, Z. Luo, L. Tan, J. Liu, R. Li, D. Wei, and Y.-J. Zhang. ESM2_amp: an interpretable framework for protein–protein interactions prediction and biological mechanism discovery. *Briefings in Bioinformatics*, 26(4), 2025.

B. E. Suzek, Y. Wang, H. Huang, P. B. McGarvey, C. H. Wu, and t. U. Consortium. UniRef clusters: a comprehensive and scalable alternative for improving sequence similarity searches. *Bioinformatics*, 2014.

D. Szklarczyk, R. Kirsch, M. Koutrouli, K. Nastou, F. Mehryary, R. Hachilif, A. L. Gable, T. Fang, N. T. Doncheva, S. Pyysalo, P. Bork, L. J. Jensen, and C. von Mering. The STRING database in 2023: protein–protein association networks and functional enrichment analyses for any sequenced genome of interest. *Nucleic Acids Research*, 2022.

J. Szymborski and A. Emad. RAPPPID: towards generalizable protein interaction prediction with AWD-LSTM twin networks. *Bioinformatics*, 38(16), 2022. ISSN 1367-4803.

K. Taha. Protein-protein interaction detection using deep learning: A survey , comparative analysis, and experimental evaluation. *Computers in Biology and Medicine*, 2025.

Z. Tang and Y. Takahashi. Analysis of Protein–Protein Interaction by Co-IP in Human Cells. pages 289–296, 2018.

G. Team, R. Anil, S. Borgeaud, Y. Wu, J. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. Dai, A. Hauth, et al. Gemini: A family of highly capable multimodal models, 2024. *arXiv preprint arXiv:2312.11805*, 10, 2025.

C. Tran, S. Khadkikar, and A. Porollo. Survey of Protein Sequence Embedding Models. *International Journal of Molecular Sciences*, 24(4), 2023. ISSN 1422-0067.

S. Tsukiyama, M. M. Hasan, S. Fujii, and H. Kurata. LSTM-PHV: prediction of human-virus protein–protein interactions by LSTM with word2vec. *Briefings in Bioinformatics*, 2021.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is All you Need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

P.-O. Vidalain, M. Boxem, H. Ge, S. Li, and M. Vidal. Increasing specificity in high-throughput yeast two-hybrid experiments. *Methods*, 32(4):363–370, Apr. 2004. ISSN 1046-2023.

X. Wang, Z. Zheng, F. Ye, D. Xue, S. Huang, and Q. Gu. Diffusion Language Models Are Versatile Protein Learners, 2024.

L. Wu, Y. Tian, Y. Huang, S. Li, H. Lin, N. V. Chawla, and S. Z. Li. MAPE-PPI: Towards Effective and Efficient Protein- Protein Interaction Prediction via Microenvironment-Aware Protein Embedding. 2024.

Z. Xiao, H. Yuan, W. Li, N. Jiang, and Y. Xia. Highly-Confident Protein Interactome Prediction via Variational Autoencoder. In *IEEE*, 2021.

F. Yang, K. Fan, D. Song, and H. Lin. Graph-based prediction of Protein-protein interactions with attributed signed graph embedding. *BMC Bioinformatics*, 2020.

Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. R. Salakhutdinov, and Q. V. Le. XLNet: Generalized Autoregressive Pretraining for Language Understanding. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d. Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

A. Zafar, N. Saba, A. Arshad, A. Alabrah, S. Riaz, M. Suleman, S. Zafar, and M. Nadeem. Convolutional Neural Networks: A Comprehensive Evaluation and Benchmarking of Pooling Layer Variants. *Symmetry*, 16, 2024. ISSN 2073-8994.

J. Zhang, J. Durham, and Qian Cong. Revolutionizing protein–protein interaction prediction with deep learning. *Current Opinion in Structural Biology*, 2024.